

QA_System_BasedOn_Local_LLM 设计与实现

2026年8月21日 7:51

项目：本地模型部署工具用 llama.cpp，开发一个简单的**智能问答系统**

整体架构：

llama.cpp 做**推理后端**（GGUF 模型） + **Python llama-cpp-python** 绑定实现对话，支持**对话历史记忆** + 模型选蒸馏推理模型 DeepSeek-R1-Distill-Qwen-1.5B-Instruct-Q4_K_M.gguf

本地**模型部署工具**：底层推理库（程序开发，自己写代码加载模型） llama.cpp

大模型**基础调用**：llama.cpp 调用

模型加载 Llama()

创建会话 Llama::create_chat_completion() 等

可尝试令一种选择：使用 llama.cpp 源码 开发问答系统（不使用 llama-cpp-python）

2种方式：

1. 调用 llama-server（HTTP 服务）：编译出官方 llama-server，作为独立后端；Python 做客户端发 HTTP 请求（最常用，推荐）

2. C/C++ 直接二次开发：基于 llama.cpp 的 C API，写 C 程序实现问答（纯 C 开发）

DeepSeek-R1-Distill-Qwen-1.5B-Q4_K_M 模型文件的问答系统，可以问哪些问题能得到答案？

蒸馏 推理模型：DeepSeek-R1 是 **深度思考 推理模型**，用大模型 R1 的 **思考样本** 蒸馏 到 **Qwen-1.5B**，小尺寸（1.5B）

主打：数学推理、逻辑题、简单代码、脑筋急转弯，会输出思考过程。

Q4_K_M 是**量化版本**，损失不大，适合**本地** CPU/GPU 跑。

1. 环境信息

(1) 电脑配置

1) Windows11 + VirtualBox + Ubuntu24.04

2) 物理内存

我的电脑 -> 属性 -> 16G

(2) Ubuntu 配置

1) 物理内存大小（8G）

free -h

-h 人类易读单位 GB/MB

total：就是分配给 Ubuntu 的 总物理内存

2) 磁盘大小（105G）

在宿主机（真实电脑 VirtualBox 界面）查看虚拟磁盘

方法1：

[1] VirtualBox 主界面 → 选中 VMLy 虚拟机 → 【设置】→ 【存储】

[2] 点击 SATA 控制器下面磁盘，右侧显示：虚拟大小（最大上限）/ 真实大小

方法2：

宿主机磁盘：找到对应的 .vdi 文件，右键看文件属性 -> 大小 = 真实硬盘占用

(3) VirtualBox 修改虚拟机内存完整步骤

重要：必须完全关机虚拟机，不能是 保存状态/挂起。

修改内存 不会破坏 Ubuntu 任何文件，内存是临时运行资源，不碰虚拟硬盘。

[1] 在 Ubuntu 虚拟机内部执行关机

[2] 在 VirtualBox 主界面，选中你的虚拟机（VMLy），点击上方 【设置】按钮。

[3] 左侧选择 【系统】→ 【主板】 标签页。

[4] 基础内存 (Base Memory)：拖动滑块或者直接输入数值，单位 MB。

- 8GiB → 填写 8192

[5] 点击 【确定】保存，启动虚拟机。

[6] 进入 Ubuntu 验证是否生效：

free -h

(4) 虚拟环境 下用 pip 网 虚拟环境中装 python 包

报错 externally-managed-environment：是 Debian/Ubuntu 23.04+/Python3.12+ 的保护机制：禁止直接用 pip 往系统 Python 装包，防止破坏系统自带 Python。

解决：用虚拟环境

0) 安装 venv 依赖（多个项目只需要在第1个项目时执行1次）

```
sudo apt update
sudo apt install python3-full python3-venv
```

1) 创建、激活 虚拟环境

```
# 进入项目目录
cd ~/AI/1_QA_System_BasedOn_Local_LLM

# 创建 venv 虚拟环境，文件夹 名字就叫 venv
python3 -m venv venv

# 激活虚拟环境：激活成功后终端前面会出现 (venv) 标记
source venv/bin/activate
```

2) 后续每次打开终端做项目，都要先执行：

```
cd ~/AI/1_QA_System_BasedOn_Local_LLM
source venv/bin/activate
```

Note:

```
退出虚拟环境
deactivate
```

3) pip 安装：

1) pip install llama-cpp-python

2) 在当前 Python 环境（你的 venv 虚拟环境），使用 指定的 pip 镜像源（清华国内镜像源），安装库：llama-cpp-python（huggingface 下载工具）
(venv) ly@VMLy:~/AI/1_QA_System_BasedOn_Local_LLM\$ pip install llama-cpp-python -i <https://pypi.tuna.tsinghua.edu.cn/simple>

测试 llama-cpp-python 是否安装成功

```
pip list | grep llama
```

正常输出示例：

```
llama-cpp-python    0.3.xx
```

NVIDIA 显卡：安装时会自动编译 CUDA 加速；

若没有 GPU，自动走 CPU 推理。

(5) 虚拟环境 **最佳实践**

1) 每个项目都要建立一个虚拟环境吗？还是多个项目可以共享一个虚拟环境？代码目录和 虚拟环境该怎么设置？

最佳实践：一个项目一个独立虚拟环境。不建议多个项目共用同一个 venv。

共用 venv 的缺点

[1] 依赖库 版本冲突（最头疼）

项目 A 需要 numpy==1.24，项目 B 需要 numpy==2.1。
共用一个 venv 只能装一个版本，必然有一个项目直接报错。

[2] 包越装越杂，每个项目的 依赖库 难区分，卸载库难对应，难复现环境。

[3] 导出依赖 pip freeze > requirements.txt 会把所有项目的包导出，混入无关库。

2) 项目文件目录

```
~/AI/1_QA_System_BasedOn_Local_LLM/
├── venv/           # 虚拟环境文件夹，在项目根目录，几个 GB
├── main.py
├── requirements.txt # （导出的）venv 的（纯文本）依赖清单，用来在别的机器 重建环境
└── 其他代码文件
```

Note:

不要手动编辑 venv 内部任何文件！venv 是自动生成的

venv 损坏：

```
删 venv 文件夹
重新 python3 -m venv venv 重建
再 pip install -r requirements.txt 恢复环境
```

删除项目代码文件夹前，要手动卸载 pip 包吗？

不需要。虚拟环境是隔离的，删 venv 文件夹，安装的包全部消失，不会影响系统 Python。

3) 环境备份：激活 venv -> 导出依赖

```
(venv) ly@pip freeze > requirements.txt
```

以后换机器 或 venv 损坏，只要重建虚拟环境，执行：

```
(venv) ly@pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple  
一键恢复全部包。
```

3) git 版本控制：

文件 .gitignore 要含 venv/ => venv/ 不会提交到 git 仓库

.gitignore 的作用：告诉 Git 忽略 某些目录/文件，不让它们被纳入版本控制。

.gitignore 示例：

.gitignore

虚拟环境

venv/

.venv/

Python缓存

__pycache__/

*.pyc

*.pyo

*.pyd

LLM大模型权重，不要上传GGUF模型文件

*.gguf

*.bin

*.safetensors

models/

1. llama.cpp

(1) llama.cpp 源码下载（不直接修改 llama.cpp 时，不必下载 llama.cpp 源码）

```
ly@VMLy:~/AI$ git clone https://github.com/ggml-org/llama.cpp.git  
Cloning into 'llama.cpp'...  
remote: Enumerating objects: 114756, done.  
remote: Counting objects: 100% (931/931), done.  
remote: Compressing objects: 100% (392/392), done.  
remote: Total 114756 (delta 729), reused 539 (delta 539), pack-reused 113825 (from 2)  
Receiving objects: 100% (114756/114756), 421.58 MiB | 1.14 MiB/s, done.  
Resolving deltas: 100% (80736/80736), done
```

(2) llama-cpp-python 安装

1) llama-cpp-python

= C/C++ 的 llama.cpp 的 Python wrapper = llama.cpp 封装成的 Python 库

写 Python 代码调用它，底层实际调用 llama.cpp

2) 在当前 Python 环境（你的 venv 虚拟环境），使用指定的 pip 镜像源（清华国内镜像源），安装库：llama-cpp-python（huggingface 下载工具）

```
(venv) ly@VMLy:~/AI/1_QA_System_BasedOn_Local_LLM$ pip install llama-cpp-python -i https://pypi.tuna.tsinghua.edu.cn/simple
```

2. 模型

(1) 概述

GGUF, GGML-Universal-Format

GGML (GG 作者 ggerganov, ML = Machine Learning)

[1] C 语言写的轻量级张量计算库 (llama.cpp 底层内核就是 GGML)

[2] 也是 旧的模型二进制文件格式

GGUF

llama.cpp 专用的 模型文件

例:

```
qwen2.5-7b-instruct.Q4_K_M.gguf
Q4: 4 比特量化
K: K-Quant
M = Medium (中等) ; 还有 S 小 (更小体积) , _L 大 (更高精度)
```

(2) 模型下载

推荐通义千问 Qwen2.5-7B-Instruct-Q4_K_M.gguf (中文效果好, 4bit 量化, 显存 / 内存友好)

下载 GGUF 文件, 放到本地目录, 如 ./models/qwen2.5-7b-instruct.Q4_K_M.gguf

1) 方法1

如 DeepSeek-R1-Distill-Qwen-1.5B-Instruct-Q4_K_M.gguf 下载

[1] 在当前 Python 环境 (你的 venv 虚拟环境), 使用指定的 pip 镜像源 (清华国内镜像源), 安装两个库: huggingface_hub (huggingface 下载工具) 和 hf_transfer (大文件加速插件)
(venv) ly@VMLy:~/AI/1_QA_System_BasedOn_Local_LLM\$
pip install huggingface_hub hf_transfer -i <https://pypi.tuna.tsinghua.edu.cn/simple>

[2] 配置 模型 下载站点

告诉 huggingface 工具: 请使用 HF_ENDPOINT 指定的镜像站点
export HF_ENDPOINT=https://hf-mirror.com

告诉 HuggingFace 用 Rust 写的高速下载组件 hf-transfer
export HF_TRANSFER=1

每次打开终端自动带镜像

```
把它写入 ~/.bashrc
echo 'export HF_ENDPOINT=https://hf-mirror.com' >> ~/.bashrc
source ~/.bashrc

echo 'export HF_TRANSFER=1' >> ~/.bashrc
source ~/.bashrc
```

[3] 模型 下载

```
hf download bartowski/DeepSeek-R1-Distill-Qwen-1.5B-GGUF \
--include "DeepSeek-R1-Distill-Qwen-1.5B-Q4_K_M.gguf" \
--local-dir ./models

--local-dir-use-symlinks True (默认行为)
真实模型文件 不是下载到指定的 --local-dir,
而是下载到 HuggingFace 的全局缓存目录: ~/.cache/huggingface/hub/,
在指定的 --local-dir 里面只生成 软链接 (symbolic link, 类似 Windows 快捷方式), 指向全局缓存里的 gguf 文件
```

把 真实完整的 gguf 实体文件直接保存到 --local-dir 指定目录 (./models)

直接在 <https://hf-mirror.com/bartowski/DeepSeek-R1-Distill-Qwen-1.5B-GGUF> 上下载 也行

通过 计算文件的本地哈希值, 并与网站给出的文件哈希值比较, 确认下载 没有丢包、文件没有损坏?
Ubuntu 本地计算文件 sha256 (系统自带工具, 无需安装)
sha256sum DeepSeek-R1-Distill-Qwen-1.5B-Q4_K_M.gguf

2) 方法2

1. <https://hf-mirror.com/models>

2. 顶部搜索框, 搜索带-GGUF 后缀的模型仓库, 例 Qwen/Qwen2.5-7B-Instruct-GGUF

找 GGUF 仓库技巧: 搜索关键词 GGUF, 优先作者 bartowski / unsloth / TheBloke

3. 点进模型仓库主页, 点击标签 Files and versions (文件和版本)。

4. 在文件列表找到后缀为 .gguf 的文件, 优先选 xxx-Q4_K_M.gguf (平衡速度与效果)。

! 不要下载 GGML 文件 (已淘汰); 不要点 git clone, 会下载几十 GB 全部量化版本!

5. 文件右侧点 向下箭头下载图标，浏览器开始下载该单个 gguf 文件。
网页下载大 GGUF (>5GB) 容易中断，大文件优先用下面命令行。

Note: 修改文件所有者

从 windows 下载，通过共享路径放到 ubuntu 上，，文件所有权是 root，先把 所有权 切回 当前用户 (ly)：

```
sudo chown ly:ly Qwen3.8-27B-UD-Q4_K_M.gguf
执行完再 ls -al, owner 变成ly ly, 普通用户才有写权限
```

国内备选模型平台 ModelScope <https://modelscope.cn/> 阿里国内平台，速度快，很多 GGUF、原版模型
ly@VMLy:~/AI/models\$ git clone https://www.modelscope.cn/nqyson/Qwen2.5-7B-Instruct-1M-Q4_K_M-GGUF.git

例：通义千问 Qwen2.5-7B-Instruct GGUF 版本仓库：Qwen/Qwen2.5-7B-Instruct-GGUF

只下载一个文件：qwen2.5-7b-instruct.Q4_K_M.gguf 即可
一定要下载 -Instruct 版本（对话微调版本）

3. llama.cpp 到底用在哪里

Python 业务代码（处理对话历史、输入输出）
→ llama-cpp-python (Python 接口)
→ llama.cpp (C++ 推理引擎)
→ GGUF 模型文件
→ 返回 AI 回答 token

4. 项目运行和环境配置

(1) 环境配置

1) 创建、激活 虚拟环境

```
# 进入项目目录
cd <~/AI/1_QA_System_BasedOn_Local_LLM>

# 创建 venv 虚拟环境，文件夹 名字就叫 venv
python3 -m venv venv

# 激活虚拟环境：激活成功后终端前面会出现 (venv) 标记
source venv/bin/activate
```

2) 重建虚拟环境，执行：

```
(venv) ly@pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
```

(2) 项目运行

```
python test.py
```

然后，交互式运行（输入 exit 回车后退出）

5. 版本

(1) 版本1：控制台对话（基础版，完整可运行）

核心点：

- [1] 用**模型官方 chat template** 构造对话消息
- [2] **维护对话历史**，实现**多轮问答**
- [3] n_gpu_layers=-1：把全部层卸载到 GPU；
CPU 运行改为 n_gpu_layers=0

问题：上下文会无限变长！

真实使用需要做 窗口裁剪：当 history 过长，丢弃最早的对话。

```
// test.py
from llama_cpp import Llama

# ===== 配置区 =====
# MODEL_PATH = "./models/Qwen3.8-27B-UD-Q4_K_M.gguf"
MODEL_PATH = "./models/DeepSeek-R1-Distill-Qwen-1.5B-Q4_K_M.gguf"
N_GPU_LAYERS = 0    # GPU全部加速；CPU请改为0
N_CTX = 1024        # 上下文窗口大小
MAX_NEW_TOKENS = 600
```

```
TEMPERATURE = 0.7
# =====

def chat_completion(llm: Llama, history: list):
    """
    :param llm: Llama实例对象，由main传入
    :param history: [{"role": "user", "content": "xxx"}, {"role": "assistant", "content": "xxx"}]
    返回模型回答字符串
    """
    output = llm.create_chat_completion(
        messages=history,
        max_tokens=MAX_NEW_TOKENS,
        temperature=TEMPERATURE,
        #stop=["<|im_end|>"], # Qwen终止符，不同模型终止符不一样
        stop=["< | end_of_sentence | >"] # deepseek
    )
    ans = output["choices"][0]["message"]["content"]
    return ans
```

```
if __name__ == "__main__":
    # 模型实例放到main内部，不再是全局对象
    llm = Llama(
        model_path=MODEL_PATH,
        n_gpu_layers=N_GPU_LAYERS,
        n_ctx=N_CTX,
        n_threads=2,
        verbose=False
    )

    print("=== llama.cpp 本地问答系统 ===")
    print("输入 exit 退出\n")
    chat_history = []
    while True:
        user_input = input("你: ")
        user_input = user_input.strip()
        if user_input.lower() == "exit":
            print("退出程序")
            break
        chat_history.append({"role": "user", "content": user_input})
        # 将main内的llm实例作为参数传给chat_completion
        answer = chat_completion(llm, chat_history)
        print(f"AI: {answer}\n")
        chat_history.append({"role": "assistant", "content": answer})
```

(2)

6. 各版本测试

(1) 第1版（终端交互式）测试

```
(venv) ly@WMLy:~/Documents/AI/1_QA_System_BasedOn_Local_LLM$ python IntelligentQASys.py
==== llama.cpp 本地问答系统 ====
输入 exit 退出
你: 1+2=7
AI: <think>
~/think>
1 + 2 = 3
你: |
```

```
ly@WMLy:~/Documents/AI/1_QA_System_BasedOn_Local_LLM$ source venv/bin/activate
(venv) ly@WMLy:~/Documents/AI/1_QA_System_BasedOn_Local_LLM$ python test.py
==== llama.cpp 本地问答系统 ====
输入 exit 退出
你: 解释什么是递归？
AI: <think>
嗯，递归是什么？我好像以前听说过，但具体是什么意思呢？让我仔细想想。

首先，递归这个词听起来有点复杂。我记得老师以前说过，递归是一种编程方法，但具体怎么操作的我还不是很清楚。也许它和函数调用有关？比如，把一个大问题分解成更小的子问题，然后再解决这些子问题，最后再把它们组合起来解决问题？

举个例子，比如计算阶乘。阶乘就是把一个数乘以所有比它小的正整数。比如，5的阶乘就是5*4*3*2*1，也就是120。那如果用递归的方法，如何分解呢？是不是可以把5的阶乘分解成5乘以4的阶乘？然后4的阶乘又分解成4乘以3的阶乘，依此类推，直到乘以1的阶乘，也就是1。这样一步步展开，直到达到基本的终止条件，也就是1的阶乘等于1。

那这个例子说明了什么呢？哦，对了，递归的关键在于函数本身定义的时候使用了自身的调用。也就是说，函数在定义的时候可能会包含递归调用，而这些递归调用又会再次调用函数本身。比如，阶乘函数的定义可以写成f(n) = n * f(n-1)，其中f(n-1)又是递归调用的f函数。

那递归在编程中的应用是什么呢？除了计算阶乘，还有没有别的例子？比如，处理字符串中的字符，或者处理数组中的元素。比如，字符串的反转可以通过递归实现，每次把第一个字符和最后一个字符交换，然后递归地处理剩下的子字符串。这样不断分解，直到剩下的子字符串为空或只有一个字符。

递归的优点是什么呢？首先，它容易理解，因为递归结构很直观，不像迭代那样有复杂的循环结构。其次，递归通常可以减少代码量，因为可以利用函数的自相似性来简化代码。比如，求最大公约数的时候，用欧几里得算法，就是一个典型的递归问题。

不过，递归也有可能带来一些问题。比如，栈溢出的问题，因为每次递归调用都要在栈上调用函数，如果深度太大，就会导致栈溢出，无法完成计算。还有，递归的效率可能不如迭代，特别是当递归深度很高的时候，因为每次调用都需要用栈存储状态，而迭代则直接在内存中执行。

那么，有没有什么方法来避免递归带来的这些问题？比如，使用递归的优化方法，比如尾递归优化，或者在代码中设置递归的限制深度，避免栈溢出。此外，可能需要检查递归的终止条件是否正确，确保在递归深度足够时才会停止，否则可能会出现死循环。

总结一下，递归
```

```
(venv) ly@ONLY:~/Documents/AI/1_08_System_BasedOn_Local_LLM$ python test.py
==== llama.cpp 本地问答系统 ====
输入 exit 退出

你：3x+5=20，求解x，给出完整解答
AI：<think>
首先，我看到方程是3x加5等于20。我的目标是解这个方程，找到x的值。

第一步是将常数项移到等式另一边。我可以从两边减去5，这样左边只剩下3x，右边则变为20减5，等于15。

接下来，我需要消去系数3。为了做到这一点，我可以两边都除以3，这样左边的3x就会变成x，右边则是15除以3，等于5。

这样，解出来的x就是5。

为了验证结果是否正确，我可以将x代入原方程，看看是否成立。3乘以5等于15，再加上5确实等于20，验证了结果的正确性。

因此，x的值是5。
</think>

要解方程  $(3x + 5 = 20)$ ，按照以下步骤进行：
```

```
**步骤 1: 移项**

首先，将常数项 (+5) 从左边移到右边：

\[\begin{aligned} 3x + 5 &= 20 \\ \end{aligned}\]
```

```
\[\begin{aligned} 3x &= 20 - 5 \\ \end{aligned}\]
```

```
\[\begin{aligned} 3x &= 15 \\ \end{aligned}\]
```

```
**步骤 2: 消去系数**

接下来，消去系数3，将两边同时除以3：
```

```
\[\begin{aligned} \frac{3x}{3} &= \frac{15}{3} \\ \end{aligned}\]
```

```
\[\begin{aligned} x &= 5 \\ \end{aligned}\]
```

```
**验证**

将  $(x = 5)$  代入原方程验证：
```

```
\[\begin{aligned} 3(5) + 5 &= 15 + 5 = 20 \\ \end{aligned}\]
```

```
\[\begin{aligned} 20 &= 20 \quad \text{\textit{成立}} \\ \end{aligned}\]
```

```
因此，方程的解是：
```

```
\[\boxed{5}\]
```

```
你：█
```

你：3x+5=20，求解x，给出完整解答

AI：<think>

首先，我看到方程是3x加5等于20。我的目标是解这个方程，找到x的值。

第一步是将常数项移到等式另一边。我可以从两边减去5，这样左边只剩下3x，右边则变为20减5，等于15。

接下来，我需要消去系数3。为了做到这一点，我可以两边都除以3，这样左边的3x就会变成x，右边则是15除以3，等于5。

这样，解出来的x就是5。

为了验证结果是否正确，我可以将x代入原方程，看看是否成立。3乘以5等于15，再加上5确实等于20，验证了结果的正确性。

因此，x的值是5。

</think>

要解方程 $(3x + 5 = 20)$ ，按照以下步骤进行：

你：█